T7. Patrones de diseño

Patrones de diseño

Los **patrones** son **soluciones** de eficiencia demostrada a problemas de diseño **comunes**. Los sistemas que hacen explícitos los patrones usados son más comprensibles y adaptables. Existen **patrones de diseño** (mecanismos) y **patrones de arquitectura** (frameworks).

Los **patrones de diseño** son *plantillas* que muestran la **estructura** y el **comportamiento** de una sociedad de clases (y objetos). Son soluciones reutilizables, probadas y eficientes a problemas de diseño que se repiten con frecuencia en el desarrollo de software. Se representan como **colaboraciones** con aspectos: **estáticos** (diagramas de clases) y **dinámicos** (diagramas de interacción).

Nota: Según el paradigma orientado a objetos, los patrones de diseño son la descripción de clases y objetos que se relacionan para resolver en un determinado contexto un problema de diseño repetido.

- <u>**Reutilización**</u>

Un diseño debe adecuarse a **requisitos** actuales y anticipar problemas futuros. Los expertos no afrontan cada situación partiendo de cero, si no que **reutilizan soluciones** útiles en el pasado. Los patrones resuelven problemas concretos de diseño y ayudan a lograr soluciones **flexibles** y **reutilizables**.

- <u>**Ventajas**</u>

Cada patrón nombra, explica y evalúa un **diseño recurrente** en sistemas orientados a objetos y permiten lograr *rápidamente* un bien diseño **reutilizando** experiencia previa. Además, ayudan a elegir **alternativas** que hacen que un sistema sea reutilizable y a evitar las que provocan el efecto contrario. Finalmente, facilitan la **documentación** y el mantenimiento de sistemas existentes.

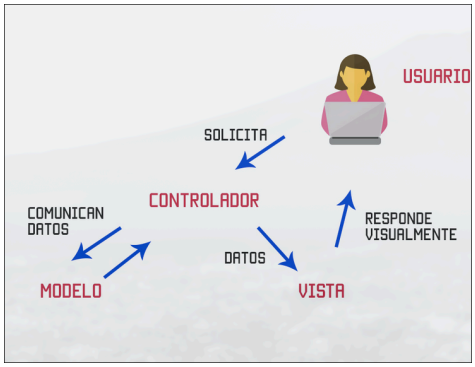
- <u>**Elementos de un patrón**</u>

- **Nombre:** descripción concisa del problema de diseño.
- **Problema**: contexto de aplicación y situación en la que usar el patrón.
- **Solución**: elementos del diseño, sus relaciones y responsabilidades.
- **Consecuencias**: ventajas e inconvenientes de aplicar el patrón.

MVC (Modelo-Vista-Controlador)

MVC es un patrón de diseño arquitectónico muy usado en interfaces gráficas y aplicaciones web. Componentes:

- **Modelo**: objeto de aplicación; contiene los datos y lógica de negocio.
- **Vista**: representación del modelo; muestra los datos del modelo al usuario.
- **Controlador**: respuesta de la interfaz a la entrada ofrecida por el usuario y actualiza el modelo o la vista.



Las vistas y los modelos se **desacoplan** usando un protocolo de suscripción/notificación (de modo que se puede cambiar el aspecto visual sin tocar la lógica. Es posible **anidar** vistas o tener múltiples representaciones del mismo modelo. La respuesta de una vista puede cambiar sin alterar su representación visual.

Clasificación de los patrones

Se pueden clasificar según su propósito:

- **De creación**: cómo se crean los objetos.
- **Estructurales**: cómo se organizan y combinan las clases y objetos para formar estructuras más grandes.
- **De comportamiento**: cómo interactúan y se comunican los objetos.

O según su ámbito:

- **De clase**: se aplican a relaciones entre clases y suelen basarse en **herencia**.
- **De objeto**: se centran en relaciones entre objetos y utilizan la **composición**.

	De creación	Estrucutales	De comportamiento
Clase	**Factory Method**	Adapter	Interpreter Template Method
Objeto	**Abstract Factory** Builder Prototype **Singleton**	Adapter Bridge **Composite** **Decorator** **Facade** Flyweight Proxy	Chain of Responsibility Command Iterator Mediator Memento **Observer** **State** Strategy Visitor

Resolución de problemas de diseño

Los patrones de diseño ayudan a la **identificación** de los objetos necesarios, a la determinación del **tamaño** de los objetos, a la programación para interfaces, a la reutilización y al diseño pensando en la evolución del sistema.

- <u>Identificación de los objetos necesarios</u>

Muchos elementos de un diseño proceden del análisis y después aparecen clases sin equivalente en el mundo real. Los patrones ayudan a identificar las abstracciones menos obvias.

- <u>Determinación de su tamaño</u>

Los objetos pueden variar notablemente en tamaño y en número. Algunos patrones de diseño describen formas concretas de descomponer un objeto en otros más pequeños.

- <u>Programación para interfaces</u>

Se debe restringir el uso de referencias a clases concretas, los objetos deben mantener un conocimiento mutuo limitado. Los **patrones** de **creación** aseguran que el sistema se diseñe en términos de interfaces.

- <u>Mecanismos de reutilización</u>

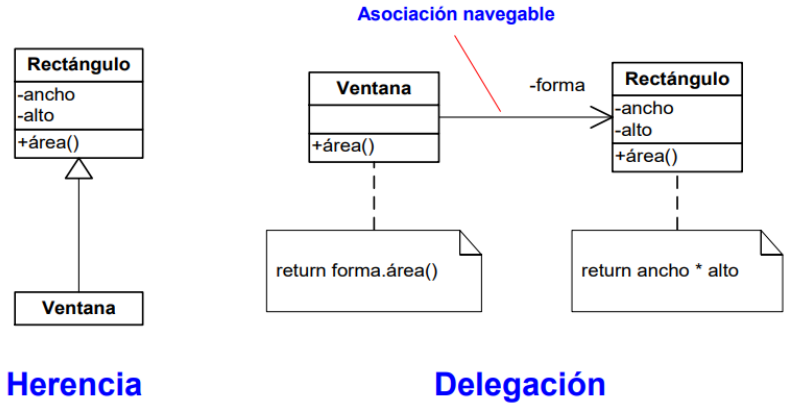
- **Herencia** (reutilización de *caja blanca*): se define en tiempo de compilación y rompe el principio de encapsulación, la clase hija obtienelos atributos y métodos de la clase padre. Se representa con Flecha y triángulo blanco.

- **Composición** (reutilización de *caja negra*): los enlaces (referencias) entre objetos se establecen en tiempo de ejecución. Cualquier otro objeto puede

ser reemplazado dinámicamente por otro del mismo tipo. El acceso a objetos a través de interfaces no rompe la encapsulación (en el ejemplo, Ventana tiene una referencia a triángulo y delega el cálculo del área en el).

Nota: **Herencia vs asociación:** según el principio DOO, se debe equilibrar el uso de la asociación y la generalización como mecanismos complementarios. Además, la herencia facilita la construcción, pero la composición aporta mayor flexibilidad.

- **Delegación**: es el modo de asociación que suple a la herencia por la cual dos objetos tratan una petición (un objeto receptor delega en su ayudante). Tiene la ventaja de que facilita la combinación de comportamientos en tiempo de ejecución, pero el inconveniente de que añade complejidad.



Diseño para el cambio

La clave para maximizar la reutilización es **anticipar** nuevos requisitos y cambios en los requisitos existentes. Los patrones aseguran que el sistema pueda **evolucionar** de forma concreta y cada patrón permite que **algún aspecto varíe** independientemente de los otros, proporcionando robustez frente a un tipo de cambio dado.

Uso de los patrones de diseño

Para seleccionar un patrón se deben examinar las causas de **rediseño** o decidir qué aspecto debería ser variable. Los patrones facilitan el mantenimiento a cambio de **complicar** el diseño (e incluso disminuir el rendimiento), sin embargo, e deben aplicar sólo si la **flexibilidad** proporcionada merece la pena.